Deuteron · 3-minute video script

359 spoken words · six visual scenes · finish speaking by 2:55, then hold for five seconds. Record one clean, continuous take with no background music. Read only the narration below; the timecodes and visual cues are for rehearsal.

0:00–0:20 | Slide 1: Scheduled IR to RTX 5090

A fast GPU still needs a good schedule. We built Deuteron to compile scheduled Emmy Kernel IR into PTX for the RTX 5090. We reduce repeated work and memory traffic while preserving each workload's original launch, inputs, and numerical tolerances.

Cue: Presenter opening, then slide 1. Pause after “good schedule.”

0:20–0:50 | Slide 2: Structure, guards, and LLVM

First, we validate the kernel and launch metadata. Custom passes then recognize reusable loads, cooperative reductions, and matrix pipelines. Each rewrite checks shapes, ownership, and dependencies; if its proof fails, supported code keeps the generic path. We lower the result into typed LLVM IR. LLVM supplies optimization and the NVPTX backend; our contribution is the GPU analysis, guarded transformations, and lowering.

Cue: Slide 2 pipeline. Give the LLVM credit its own breath.

0:50–1:25 | Slide 3: Tensor cores, copies, and types

For matrix multiplication, we use compensated BF16x3 or TF32x3 tensor products. Buffers and accumulation stay FP32; lower precision is internal, and these modes remain approximations. Ordinary FP32 multiplication is available too. Asynchronous global-to-shared copies, with explicit waits and barriers, let computation overlap data movement. A separate FP16 storage path provides exact widening and nearest-even narrowing, with explicit FP32 arithmetic. The focus is reducing memory traffic and feeding the hardware effectively.

Cue: Slide 3 hardware columns. Say the tensor names slowly.

Deuteron · demo, results, and close

1:25–1:50 | Slide 4: Compile it, inspect it

Let's compile the standalone matrix example in TF32x3 mode for the 5090. Compilation runs on the CPU, using only the scheduled IR and its adjacent launch metadata. The generated PTX contains mma.sync tensor instructions and cp.async copies. These are actual compiler outputs. This command only compiles; GPU correctness is tested separately.

Cue: Slide 4, real PTX capture. Leave both instruction lines visible.

1:50–2:30 | Slide 5: Measured on the open set

Across fifteen open workloads, Deuteron's geometric mean warm latency is three point seven five microseconds. That's a one point six six times speedup over generated CUDA compiled with NVCC, and one point one five times over the tested Triton baseline. We used two hundred seventy calls with balanced execution order, checking every output against the original references and tolerances. Raw samples and artifacts are included. Compilation, assembly, and network time are excluded. These are warm kernel timings on this set, with a limited Triton search.

Cue: Slide 5 chart. Slow for the ratios; keep the warm and baseline qualifiers.

2:30–2:55 | Slide 6: Reliability and an explicit scope

We also ship an installable command, restricted parsing, validated interfaces, and corruption-tolerant caches. All one hundred and three tests passed with GPU checks enabled on H100; eleven feature examples passed on the 5090. Scope stays explicit: supported scheduled IR and documented numerical modes. Deuteron makes GPU compiler decisions visible, reproducible, and measurable.

Cue: Slide 6 and presenter close. Separate the H100 and RTX test counts.

2:55–3:00: stop speaking; hold slide 6. Leave extra silence on the raw recording for editing. The full recording setup, pronunciation notes, optional cuts, and edit plan are in the companion Markdown files.